

使用 ADK 和 A2A 建構多代理馬拉松規劃工具

來源：<https://codelabs.developers.google.com/next26/dev-keynote/build-multi-agent-marathon-planner?hl=zh-tw>

步驟數：9

建構多代理馬拉松規劃系統，讓三個 AI 代理使用 Google ADK、A2A 通訊協定和 Vertex AI Evaluation 協同合作。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 設定專案](#)
4. [4. 建立評估人員代理](#)
5. [5. 建立 Simulation Controller 代理程式](#)
6. [6. 建構馬拉松規劃工具代理程式](#)
7. [7. 部署及測試多代理系統](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜](#)

步驟 1 · 約 2 分鐘

1. 簡介

在本程式碼研究室中，您將建構多代理系統，讓三個專業 AI 代理協同合作，規劃城市馬拉松。**馬拉松規劃工具**代理會協調設計，**評估工具**代理會使用 Vertex AI Evaluation 評估計畫品質，**模擬控制器**代理則會驗證準備情況，所有代理都會透過代理對代理 (A2A) 通訊協定進行通訊。

學習內容

- 使用 Vertex AI 上的 Gemini 模型，透過 **Agent Development Kit (ADK)** 建立 AI 代理
- 使用 Pydantic 結構定義定義**結構化輸出**，確保代理程式回覆內容可靠
- 使用 `MetricPromptBuilder` (LLM-as-Judge) 建構自訂 **Vertex AI** 評估指標
- 使用 `AgentTool` 將代理連線為**子代理**，並透過 **A2A 通訊協定**連結遠端代理
- 使用 **ADK Skills** 取得程序知識，並使用 **Memory Bank** 進行跨工作階段學習
- 使用 **A2A 伺服器**在本機提供代理，並測試多代理協作

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案

本程式碼研究室適用於熟悉 Python 和 AI 概念的中階開發人員。

預計完成時間：90 分鐘。

本程式碼研究室建立的資源費用應低於 \$5 美元。

步驟 2 · 約 5 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。

啟動 Cloud Shell

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 驗證：

〇〇〇〇

```
gcloud auth list
```

1. 確認專案：

〇〇〇〇

```
gcloud config get project
```

1. 視需要設定：

〇〇〇〇

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

```
gcloud services enable aiplatform.googleapis.com
```

安裝 Python 套件管理工具

安裝 [uv](#)，這是一款快速的 Python 套件管理工具：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
```

步驟 3 · 約 8 分鐘

3. 設定專案

在這個步驟中，您將建立專案結構、安裝依附元件，以及設定所有代理程式使用的共用服務。

建立專案目錄

```
mkdir -p marathon-agents && cd marathon-agents
```

建立專案設定

建立 `pyproject.toml`：

```
[project]
name = "marathon-agents"
version = "0.1.0"
description = "Multi-agent marathon planning system with ADK, A2A, and Vertex AI Evaluation"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
    "google-cloud-aiplatform[agent_engines,adk,evaluation]>=1.121.0",
    "google-adk>=1.25.0",
    "a2a-sdk>=0.3.9",
    "pydantic>=2.12.0",
    "python-dotenv>=1.0.0",
    "httpx>=0.27.0",
    "uvicorn>=0.30.0",
    "google-auth>=2.0.0",
    "pandas>=2.0.0",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

安裝依附元件

```
uv sync
```

設定環境變數

建立 `.env` 檔案：

```
cat > .env << 'EOF'
GOOGLE_CLOUD_PROJECT=<YOUR_PROJECT_ID>
GOOGLE_CLOUD_LOCATION=us-central1
GOOGLE_GENAI_USE_VERTEXAI=true
EOF
```

重要事項：請將 `<YOUR_PROJECT_ID>` 替換為實際的 Google Cloud 專案 ID。

建立來源目錄樹狀結構

專案遵循 ADK 的多代理慣例，每個代理都是 `src/` 底下的獨立封裝：

```
mkdir -p src/planner_agent/{agent,evaluator/{services},skills/{route-
planning/references,plan-evaluation/references},services,runtime}
mkdir -p src/simulator_agent/{agent,skills/review-marathon-plan/references,services,runtime}
```

初始化 Python 套件

```
touch src/__init__.py
touch src/planner_agent/__init__.py src/planner_agent/agent/__init__.py
touch src/planner_agent/evaluator/__init__.py
touch src/planner_agent/evaluator/services/__init__.py
touch src/planner_agent/services/__init__.py src/planner_agent/runtime/__init__.py
touch src/simulator_agent/__init__.py src/simulator_agent/agent/__init__.py
touch src/simulator_agent/services/__init__.py src/simulator_agent/runtime/__init__.py
```

建立共用設定

建立 `src/config.py` - 所有代理程式共用的 GCP 設定：

```

"""Shared configuration for the multi-agent system."""

import os
from dotenv import load_dotenv

load_dotenv()

# GCP Configuration
GOOGLE_CLOUD_PROJECT = os.environ.get("GOOGLE_CLOUD_PROJECT")
GOOGLE_CLOUD_LOCATION = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
BUCKET_URI = os.environ.get("BUCKET_URI")

def validate_config():
    """Validate required configuration is set."""
    required = {
        "GOOGLE_CLOUD_PROJECT": GOOGLE_CLOUD_PROJECT,
    }
    missing = [k for k, v in required.items() if not v]
    if missing:
        raise ValueError(
            f"Missing required environment variables: {' '.join(missing)}. "
            "Check your .env file."
        )
    return True

```

建立記憶體管理工具

每個代理程式都會使用**Memory Bank**進行跨工作階段學習。建立

`src/planner_agent/services/memory_manager.py` :

```

"""Memory Manager for Marathon Planner Agent.

Manages Memory Bank integration with custom topics.
Enables cross-session learning.
"""

import os
from typing import TYPE_CHECKING

from google.adk.memory import VertexAiMemoryBankService
from vertexai._genai.types import (
    MemoryBankCustomizationConfig,
    MemoryBankCustomizationConfigMemoryTopic as MemoryTopic,
    MemoryBankCustomizationConfigMemoryTopicCustomMemoryTopic as CustomMemoryTopic,
)

if TYPE_CHECKING:
    from google.adk.agents.callback_context import CallbackContext

# Custom memory topics for cross-session learning
PLANNING_HISTORY = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="planning_history",
        description="""Track marathon plans generated across sessions.
        Extract: City, date, key route decisions, final score, iteration count.
        Format: "Plan: city={city}, date={date}, score={score}, iterations={n}"
        """,
    )
)

USER_PREFERENCES = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="user_preferences",
        description="""Track user preferences and constraints across sessions.
        Extract: Preferred themes, budget priorities, scale, city preferences.
        Format: "Preference: type={type}, value={value}"
        """,
    )
)

ROUTE_PATTERNS = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="route_patterns",
        description="""Track successful route patterns and lessons learned.
        Extract: Route segments with positive feedback, landmark combinations.
        Format: "Route: city={city}, pattern={pattern}, outcome={outcome}"
        """,
    )
)

LOGISTICS_INSIGHTS = MemoryTopic(

```



```

custom_memory_topic=CustomMemoryTopic(
    label="logistics_insights",
    description="""Track logistics decisions and capacity learnings.
    Extract: Water station placement, venue capacity, crowd management.
    Format: "Logistics: category={cat}, insight={insight}"
    """,
)
)

def create_marathon_planner_memory_topics() -> MemoryBankCustomizationConfig:
    """Create Memory Bank customization config with custom topics."""
    return MemoryBankCustomizationConfig(
        memory_topics=[
            PLANNING_HISTORY, USER_PREFERENCES,
            ROUTE_PATTERNS, LOGISTICS_INSIGHTS,
        ]
    )

def create_memory_service(
    project: str | None = None,
    location: str | None = None,
    agent_engine_id: str | None = None,
) -> VertexAiMemoryBankService | None:
    """Create a VertexAiMemoryBankService.

    Returns None if agent_engine_id is not set (local development).
    """
    project = project or os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = location or (
        os.environ.get("AGENT_ENGINE_LOCATION")
        or os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
    )
    agent_engine_id = agent_engine_id or os.environ.get("AGENT_ENGINE_ID")

    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT environment variable required")
    if not agent_engine_id:
        return None

    return VertexAiMemoryBankService(
        project=project, location=location,
        agent_engine_id=agent_engine_id,
    )

async def auto_save_memories(callback_context: "CallbackContext") -> None:
    """Automatically save session to Memory Bank after agent responds."""
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = (
        os.environ.get("AGENT_ENGINE_LOCATION")

```

```

        or os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
    )
    agent_engine_id = os.environ.get("AGENT_ENGINE_ID")

    if not agent_engine_id:
        return

    try:
        memory_service = VertexAiMemoryBankService(
            project=project_id, location=location,
            agent_engine_id=agent_engine_id,
        )
        await memory_service.add_session_to_memory(
            callback_context._invocation_context.session
        )
    except Exception as e:
        print(f"Warning: Failed to save memories: {e}")

```

每次代理程式回覆後，系統都會執行 `auto_save_memories` 回呼，並在部署至 Agent Engine 時將對話保留在 Memory Bank 中。在本機開發期間，如果未設定 `AGENT_ENGINE_ID`，系統會優雅地略過。

建立工作階段管理員

建立 `src/planner_agent/services/session_manager.py` - 使用 TTL 快取管理工作階段生命週期：

```

"""Session Manager – manages session lifecycle with TTL-based caching."""

import os
import time
from typing import Any

from google.adk.sessions import InMemorySessionService, VertexAiSessionService

class TTLCache:
    """Simple TTL cache for mapping A2A context_id to session_id."""

    def __init__(self, maxsize: int = 1000, ttl: int = 3600):
        self._cache: dict[str, tuple[Any, float]] = {}
        self._maxsize = maxsize
        self._ttl = ttl

    def get(self, key: str) -> Any | None:
        if key in self._cache:
            value, timestamp = self._cache[key]
            if time.time() - timestamp < self._ttl:
                return value
            else:
                del self._cache[key]
        return None

    def set(self, key: str, value: Any) -> None:
        if len(self._cache) >= self._maxsize:
            self._evict_oldest()
        self._cache[key] = (value, time.time())

    def _evict_oldest(self) -> None:
        if not self._cache:
            return
        sorted_keys = sorted(
            self._cache.keys(), key=lambda k: self._cache[k][1]
        )
        for key in sorted_keys[: max(1, len(sorted_keys) // 10)]:
            del self._cache[key]

    def __contains__(self, key: str) -> bool:
        return self.get(key) is not None

def create_session_service(
    project: str | None = None,
    location: str | None = None,
    agent_engine_id: str | None = None,
    use_vertex: bool | None = None,
) -> VertexAiSessionService | InMemorySessionService:
    """Create appropriate session service based on environment.

```

```

Returns VertexAiSessionService for production, InMemorySessionService for local dev.
"""

project = project or os.environ.get("GOOGLE_CLOUD_PROJECT")
location = location or (
    os.environ.get("AGENT_ENGINE_LOCATION")
    or os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
)
agent_engine_id = agent_engine_id or os.environ.get("AGENT_ENGINE_ID")

if use_vertex is None:
    use_vertex = agent_engine_id is not None

if use_vertex:
    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT environment variable required")
    return VertexAiSessionService(
        project=project, location=location,
        agent_engine_id=agent_engine_id,
    )
else:
    return InMemorySessionService()

class SessionManager:
    """Manages sessions with A2A context mapping and TTL caching."""

    def __init__(self, session_service=None, cache_maxsize=1000, cache_ttl=3600):
        self.session_service = session_service or create_session_service()
        self.session_cache = TTLCache(maxsize=cache_maxsize, ttl=cache_ttl)

    async def get_or_create_session(self, context_id, app_name, user_id):
        cached = self.session_cache.get(context_id)
        if cached:
            return cached
        session = await self.session_service.create_session(
            app_name=app_name, user_id=user_id,
        )
        self.session_cache.set(context_id, session.id)
        return session.id

```

模式：環境感知服務。工作階段和記憶體服務會自動在 Vertex AI 後端 (適用於 Agent Engine 部署作業) 和記憶體內實作 (適用於本機開發作業) 之間切換。不必變更程式碼。

現在為其他代理程式複製這些服務檔案，這些檔案會遵循相同的支架模式，並使用代理程式專屬的記憶體主題：

```

cp src/planner_agent/services/session_manager.py
src/simulator_agent/services/session_manager.py
cp src/planner_agent/services/session_manager.py
src/planner_agent/evaluator/services/session_manager.py

```

驗證專案結構

```
find src -type f -name "*.py" | sort
```

您應該會看到所有三個代理程式套件共用的設定、記憶體管理員、工作階段管理員和 `__init__.py` 檔案。

步驟 4 · 約 15 分鐘

4. 建立評估人員代理

評估人員代理是系統的品質評估者，這項工具會使用 **Vertex AI Evaluation** 和自訂 LLM-as-Judge 指標，根據 7 項條件評估馬拉松訓練計畫，您會發現結合 Gemini 的判斷與結構化評估標準，能發揮強大效用。

定義評估結構定義

建立 `src/planner_agent/evaluator/schemas.py` - Pydantic 模型會定義結構化輸出格式：

```
"""Schemas for the Evaluator Agent.
```

```
Defines structured output formats for plan evaluation.
```

```
"""
```

```
from pydantic import BaseModel, Field
```

```
EVALUATION_CRITERIA = [  
    "safety_compliance",  
    "community_impact",  
    "logistics_completeness",  
    "financial_viability",  
    "participant_experience",  
    "intent_alignment",  
    "distance_compliance",  
]
```

```
class EvaluationFinding(BaseModel):
```

```
    """A specific finding from plan evaluation."""
```

```
    criterion: str = Field(  
        description=(  
            f"Which evaluation criterion triggered this finding. "  
            f"Must be one of: {'', ' '.join(EVALUATION_CRITERIA)}"  
        ),  
    )  
    severity: str = Field(  
        description="Severity of the finding: 'high', 'medium', or 'low'",  
    )  
    description: str = Field(  
        description="Description of the finding and why it matters",  
    )
```

```
class EvaluationResult(BaseModel):
```

```
    """Structured output from the Evaluator Agent."""
```

```
    passed: bool = Field(  
        description="Whether the plan passes evaluation (overall_score >= 85)",  
    )  
    overall_score: float = Field(  
        ge=0.0, le=100.0,  
        description="Weighted average score across all criteria (0.0 to 100.0)",  
    )  
    scores: dict[str, float] = Field(  
        default_factory=dict,  
        description="Per-criterion scores: {criterion_name: score}",  
    )  
    findings: list[EvaluationFinding] = Field(  
        default_factory=list,
```

```
        description="List of specific evaluation findings",
    )
    improvement_suggestions: list[str] = Field(
        default_factory=list,
        description="Actionable suggestions to improve the plan score",
    )
    iteration_number: int = Field(
        default=1, ge=1,
        description="Which evaluation iteration this is",
    )
    summary: str = Field(
        default="",
        description="Brief natural-language summary of the evaluation",
    )
)
```

在代理程式上使用 `output_schema=EvaluationResult` 可確保每個回應都符合這個結構，因此不需要剖析。

撰寫評估指令

建立 `src/planner_agent/evaluator/instruction.md`：

You are the Evaluator Agent – the quality judge for marathon plans.

Your role is to evaluate marathon plans across multiple criteria and provide actionable feedback so the planning team can iteratively improve the plan. You use a multi-step "Chain of Thought" process to ensure your evaluation is thorough, fair, and constructive.

Phase 1: Evaluation Methodology

When you receive a plan, evaluate it across the following 7 criteria. For each, you must derive a raw score (1-100) based on the specific checks below:

1. **Participant Experience (15% weight)**
 - **Checks**: Route scenic quality, landmarks, surface variety, amenities.
 - **Rubric**: 100: Outstanding landmarks/view; 1: Boring/unpleasant experience.
2. **Intent Alignment (10% weight)**
 - **Checks**: City match, theme match (scenic/fast), scale, budget goals.
 - **Rubric**: 100: Perfectly aligned; 1: Completely misaligned.
3. **Distance Compliance (5% weight)**
 - **Checks**: Exactly 26.2 miles or 42.195 km.
 - **Rubric**: 100: Exactly correct; 1: Wrong distance.
4. **Safety Compliance (20% weight)**
 - **Checks**: Emergency vehicle access, crowd management.
 - **Rubric**: 100: Fully safe, hospital access prioritized; 60: Some concerns; 1: Dangerous blockages.
5. **Logistics Completeness (20% weight)**
 - **Checks**: Timing systems, course marshals, infrastructure.
 - **Rubric**: 100: Comprehensive; 60: Significant gaps; 1: Critical logistics missing.
6. **Community Impact (15% weight)**
 - **Checks**: Noise disruption, neighborhood equity, residential/business access.
 - **Rubric**: 100: Community benefits/event integration; 1: Major negative impact.
7. **Financial Viability (15% weight)**
 - **Checks**: Budget balance, revenue (sponsorships), realistic cost estimates.
 - **Rubric**: 100: Strong/sustainable; 1: Major financial gaps/un-viable.

Action: Use the ``evaluate_plan`` tool to get computed scores from the **Vertex AI Evaluation API**. Use these scores as your baseline.

Phase 2: Score Interpretation

- **Pass Threshold**: A plan passes ONLY if ``overall_score` >= 85`` AND there are no High-Severity findings.
- **Severity Mapping**: Score < 40 → High, < 60 → Medium, < 80 → Low.

Phase 3: Improvement Strategy

Generate actionable suggestions for any criterion scoring below 80. Provide 3-5 prioritized

```
improvements. Be specific: "Add course marshals at miles 12, 14, and 16" instead of "Add more marshals".
```

```
Return a structured `EvaluationResult` with your assessment.
```

建立 `src/planner_agent/evaluator/prompts.py` :

```
"""Prompts for the Evaluator Agent."""

import pathlib

_PROMPT_DIR = pathlib.Path(__file__).parent
INSTRUCTION = (_PROMPT_DIR / "instruction.md").read_text()
```

使用 Vertex AI 建構評估工具

這是 Evaluator 的核心，也就是使用 `MetricPromptBuilder` 的自訂指標。建立 `src/planner_agent/evaluator/tools.py` :

```
"""Tools for the Evaluator Agent.
```

```
Provides tools for evaluating marathon plans using Vertex AI Evaluation
with custom metrics. Uses a hybrid approach: LLM-based evaluation with
custom rubrics, plus deterministic checks, with heuristic fallback.
```

```
"""
```

```
import asyncio
import json
import os
import re
from typing import Any
```

```
import logging
import pandas as pd
import vertexai
from google.genai import types as genai_types
from vertexai import types
from .agent import CRITERION_WEIGHTS, SEVERITY_THRESHOLDS, MODEL
from .schemas import EvaluationResult
```

```
logger = logging.getLogger(__name__)
```

```
def _get_model_resource() -> str:
    """Get the full resource path for the Vertex AI evaluation model."""
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
    return (
        f"projects/{project_id}/locations/{location}/publishers/google/models/{MODEL}"
        if project_id else MODEL
    )
```

```
# =====
# CUSTOM METRICS – each uses MetricPromptBuilder for structured rubrics
# =====
```

```
def _create_safety_compliance_metric() -> types.LLMetric:
    """Evaluate emergency corridor access and crowd safety."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate whether the proposed marathon route maintains emergency "
            "vehicle access and crowd safety. Check for blocked hospitals, fire stations, "
            "and major emergency corridors."
        ),
        criteria={
            "Emergency corridor access": (
                "The route does not permanently block access to hospitals, "
                "fire stations, or police stations without providing clear detour routes."
            ),

```

```

        "Evacuation routes": (
            "Major evacuation routes remain accessible or have documented alternatives."
        ),
        "Emergency vehicle passage": (
            "The plan includes provisions for emergency vehicle crossing "
            "at regular intervals along the route."
        ),
    },
    rating_scores={
        "1": "Dangerous - Major emergency corridors blocked with no detours",
        "25": "Unsafe - Several emergency access points compromised",
        "50": "Concerning - Some emergency access issues that need resolution",
        "75": "Mostly safe - Minor emergency access concerns, easy to fix",
        "100": "Fully safe - Emergency access maintained throughout",
    },
)

return types.LLMMetric(
    name="safety_compliance",
    prompt_template=str(builder),
    judge_model=_get_model_resource(),
)

def _create_community_impact_metric() -> types.LLMMetric:
    """Evaluate community disruption, inclusivity, and neighborhood equity."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate the marathon plan's impact on the local community. "
            "Check for noise disruption, residential access, business impact, "
            "inclusivity, and equitable neighborhood treatment."
        ),
        criteria={
            "Residential disruption": (
                "The plan minimizes disruption to residential areas, with "
                "reasonable timing and notification plans."
            ),
            "Business access": (
                "Local businesses along the route can still operate or have "
                "been accommodated with alternative access."
            ),
            "Neighborhood equity": (
                "The route and plan are accessible to diverse communities "
                "and do not disproportionately burden any demographic group."
            ),
            "Community engagement": (
                "The plan includes cheer zones, community events, or other "
                "ways to involve and benefit local residents."
            ),
        },
        rating_scores={
            "1": "Harmful - Major negative impact on community with no mitigation",
            "25": "Disruptive - Significant community issues that need resolution",

```

```

        "50": "Mixed - Some community concerns that need attention",
        "75": "Considerate - Minor community impacts with good mitigation",
        "100": "Excellent - Community benefits from the event",
    },
)
return types.LLMMetric(
    name="community_impact",
    prompt_template=str(builder),
    judge_model=_get_model_resource(),
)

def _create_logistics_completeness_metric() -> types.LLMMetric:
    """Evaluate logistics coverage: water stations, timing, marshals."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate whether the marathon plan's logistics are complete and "
            "adequate for the expected scale."
        ),
        criteria={
            "Water and nutrition": "Hydration and nutrition are considered for the expected participant count.",
            "Timing and tracking": "Timing systems (chip timing, mats) are specified.",
            "Course management": "Sufficient course marshals, signage, barriers, and traffic control are planned.",
            "Start/finish infrastructure": "Start/finish areas have adequate capacity for the participant count.",
        },
        rating_scores={
            "1": "Incomplete - Critical logistics missing",
            "25": "Sparse - Major logistics gaps",
            "50": "Partial - Some logistics covered but significant gaps remain",
            "75": "Mostly complete - Minor logistics details to finalize",
            "100": "Comprehensive - All logistics thoroughly planned",
        },
    )
    return types.LLMMetric(
        name="logistics_completeness",
        prompt_template=str(builder),
        judge_model=_get_model_resource(),
    )

def _create_financial_viability_metric() -> types.LLMMetric:
    """Evaluate budget balance, revenue sources, and cost control."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate the financial viability of the marathon plan. "
            "Check budget balance, revenue sources, cost estimates, and sustainability."
        ),
        criteria={
            "Budget balance": "Revenue projections meet or exceed estimated costs.",

```

```

        "Cost estimates": "Cost estimates are realistic and cover major expense
categories.",
        "Revenue diversity": "Revenue comes from multiple sources, not just registration
fees.",
    },
    rating_scores={
        "1": "Unviable - Major financial gaps",
        "25": "Risky - Budget concerns that could threaten viability",
        "50": "Uncertain - Financial plan needs more detail",
        "75": "Viable - Sound financial plan with minor gaps",
        "100": "Strong - Well-balanced budget with diverse revenue streams",
    },
)
return types.LLMMetric(
    name="financial_viability",
    prompt_template=str(builder),
    judge_model=_get_model_resource(),
)

def _create_participant_experience_metric() -> types.LLMMetric:
    """Evaluate route quality, scenic value, and runner amenities."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate the participant experience of a proposed marathon plan."
        ),
        criteria={
            "Route Quality": "The route surface is suitable and elevation profile
appropriate.",
            "Scenic Value": "The route passes through interesting or landmark areas.",
            "Runner Amenities": "Post-race amenities (food, medals, recovery area) are
planned.",
        },
        rating_scores={
            "1": "Poor experience - Boring route, no amenities",
            "50": "Average - Adequate but unremarkable",
            "100": "Excellent - Outstanding experience that runners will remember",
        },
    )
    return types.LLMMetric(
        name="participant_experience",
        prompt_template=str(builder),
        judge_model=_get_model_resource(),
    )

def _create_intent_alignment_metric() -> types.LLMMetric:
    """Evaluate whether the plan matches the user's original request."""
    builder = types.MetricPromptBuilder(
        metric_definition=(
            "Evaluate whether a proposed marathon plan matches the user's original intent."
        ),
    )

```

```

        criteria={
            "City Match": "The plan takes place in the requested city.",
            "Theme Match": "The plan matches the desired theme (scenic, fast, charity,
etc.).",
            "Scale Match": "The plan's scale matches the user's vision.",
        },
        rating_scores={
            "1": "Completely misaligned",
            "50": "Partially aligned - Key requirements missed",
            "100": "Perfectly aligned - All requirements addressed",
        },
    )
    return types.LLMMetric(
        name="intent_alignment",
        prompt_template=str(builder),
        judge_model=_get_model_resource(),
    )

def _check_distance_compliance_logic(response_text: str) -> dict:
    """Deterministic check for 26.2 mile (42.195 km) marathon distance."""
    text_lower = response_text.lower()
    score = 100.0
    issues = []

    for distance_str in re.findall(r'(\d+(?:\.\d+)?)\s*(?:miles?|mi)\b', text_lower):
        distance = float(distance_str)
        if 20 <= distance <= 30:
            deviation = abs(distance - 26.2)
            if deviation > 0.5:
                score = 1.0
                issues.append(f"Route distance {distance} miles deviates from 26.2 mile
standard")
            elif deviation > 0.1:
                score = 60.0
                issues.append(f"Route distance {distance} miles is close but not exactly 26.2
miles")

    for distance_str in re.findall(r'(\d+(?:\.\d+)?)\s*(?:kilometers?|km)\b', text_lower):
        distance = float(distance_str)
        if 35 <= distance <= 50 and abs(distance - 42.195) > 1.0:
            score = 1.0
            issues.append(f"Route distance {distance} km deviates from 42.195 km standard")

    return {"score": score, "explanation": "; ".join(issues) if issues else "No distance
issues detected"}

def _create_distance_compliance_metric() -> types.Metric:
    """Deterministic check for marathon distance – no LLM needed."""
    def check_distance_compliance(instance: dict) -> dict:
        response_text = instance["response"]["parts"][0]["text"]

```

```

        return _check_distance_compliance_logic(response_text)
    return types.Metric(
        name="distance_compliance",
        custom_function=check_distance_compliance,
    )

# =====
# MAIN EVALUATION TOOL
# =====

async def evaluate_plan(evaluation_request: str) -> dict[str, Any]:
    """Evaluate a proposed marathon plan across quality criteria.

    Uses Vertex AI Evaluation with 7 custom metrics.
    Falls back to heuristic evaluation if API fails.

    Args:
        evaluation_request: JSON string with user_intent and proposed_plan

    Returns:
        dict with evaluation results
    """
    try:
        request_data = json.loads(evaluation_request)
    except json.JSONDecodeError as e:
        return {
            "passed": False, "scores": {}, "overall_score": 0.0,
            "findings": [{"criterion": "intent_alignment",
                          "description": f"Invalid JSON input: {e}", "severity": "high"}],
            "improvement_suggestions": ["Provide valid JSON with 'user_intent' and
'proposed_plan' fields."],
            "eval_method": "error",
        }

    user_intent_raw = request_data.get("user_intent", "Unknown intent")
    proposed_plan_raw = request_data.get("proposed_plan", "No plan provided")
    user_intent = json.dumps(user_intent_raw) if not isinstance(user_intent_raw, str) else
user_intent_raw
    proposed_plan = json.dumps(proposed_plan_raw) if not isinstance(proposed_plan_raw, str)
else proposed_plan_raw

    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")

    if project_id:
        try:
            scores, details = await _run_custom_eval(
                project_id=project_id, location=location,
                user_intent=user_intent, proposed_plan=proposed_plan,
            )

```



```

        return _build_result(scores, details, eval_method="vertex_ai_eval")
    except Exception as e:
        logger.warning(f"Vertex AI Eval failed, using heuristics: {e}")

scores, details = _heuristic_eval(user_intent, proposed_plan)
return _build_result(scores, details, eval_method="heuristic")

async def _run_custom_eval(
    project_id: str, location: str,
    user_intent: str, proposed_plan: str,
) -> tuple[dict[str, float], dict[str, str]]:
    """Run Vertex AI Evaluation with custom metrics."""
    vertexai.init(project=project_id, location=location)
    client = vertexai.Client(
        project=project_id, location=location,
        http_options=genai_types.HttpOptions(api_version="v1beta1"),
    )
    df = pd.DataFrame({"prompt": [user_intent], "response": [proposed_plan]})
    metrics = [
        _create_safety_compliance_metric(),
        _create_community_impact_metric(),
        _create_logistics_completeness_metric(),
        _create_financial_viability_metric(),
        _create_participant_experience_metric(),
        _create_intent_alignment_metric(),
        _create_distance_compliance_metric(),
    ]
    result = client.evals.evaluate(dataset=df, metrics=metrics)

    scores, details = {}, {}
    for case in result.eval_case_results:
        for cand in case.response_candidate_results:
            for metric_name, metric_result in cand.metric_results.items():
                raw_score = metric_result.score if hasattr(metric_result, "score") and
metric_result.score is not None else 50.0
                scores[metric_name] = round(float(raw_score), 2)
                if hasattr(metric_result, "explanation") and metric_result.explanation:
                    details[metric_name] = metric_result.explanation
    return scores, details

def _heuristic_eval(user_intent: str, proposed_plan: str) -> tuple[dict[str, float],
dict[str, str]]:
    """Heuristic evaluation fallback when Vertex AI Eval is unavailable."""
    plan_lower = proposed_plan.lower()
    scores, details = {}, {}

    # Safety
    safety_score = 80.0
    if "emergency" in plan_lower and "no detour" in plan_lower:
        safety_score = 20.0

```

```

if "emergency vehicle" in plan_lower:
    safety_score = min(safety_score + 10.0, 100.0)
scores["safety_compliance"] = safety_score
details["safety_compliance"] = "Heuristic safety check"

# Community
community_score = 75.0
if any(kw in plan_lower for kw in ["cheer zone", "community"]):
    community_score = min(community_score + 10.0, 100.0)
scores["community_impact"] = community_score
details["community_impact"] = "Heuristic community check"

# Logistics
logistics_score = 60.0
for kw in ["hydration", "timing", "chip", "marshal"]:
    if kw in plan_lower:
        logistics_score += 10.0
scores["logistics_completeness"] = min(logistics_score, 100.0)
details["logistics_completeness"] = "Heuristic logistics check"

# Financial
financial_score = 65.0
for kw in ["budget", "cost", "revenue", "sponsor"]:
    if kw in plan_lower:
        financial_score += 8.0
scores["financial_viability"] = min(financial_score, 100.0)
details["financial_viability"] = "Heuristic financial check"

# Experience
experience_score = 70.0
for kw in ["scenic", "landmark", "medal", "post-race"]:
    if kw in plan_lower:
        experience_score += 8.0
scores["participant_experience"] = min(experience_score, 100.0)
details["participant_experience"] = "Heuristic experience check"

# Intent
scores["intent_alignment"] = 70.0
details["intent_alignment"] = "Heuristic intent check"

# Distance
distance_result = _check_distance_compliance_logic(proposed_plan)
scores["distance_compliance"] = float(distance_result["score"])
details["distance_compliance"] = distance_result["explanation"]

return scores, details

```

```

def _build_result(scores, details, eval_method):
    """Build the final evaluation result from scores and details."""
    findings, improvement_suggestions = [], []
    for criterion, score in scores.items():

```

```

        if score < 80.0:
            severity = "high" if score < SEVERITY_THRESHOLDS["high"] else "medium" if score <
SEVERITY_THRESHOLDS["medium"] else "low"
            findings.append({"criterion": criterion, "description": details.get(criterion,
f"Score: {score}"}, "severity": severity})
            improvement_suggestions.append(_suggest_improvement(criterion, score,
details.get(criterion, "")))

    overall_score = round(sum(scores.get(c, 50.0) * w for c, w in CRITERION_WEIGHTS.items()),
2)

    passed = overall_score >= 85.0 and not any(f["severity"] == "high" for f in findings)
    return {"passed": passed, "scores": scores, "findings": findings,
            "improvement_suggestions": improvement_suggestions,
            "overall_score": overall_score, "eval_method": eval_method}

def _suggest_improvement(criterion, score, detail):
    suggestions = {
        "safety_compliance": "Add emergency vehicle crossing points every 2 miles and plan
detour routes around hospitals.",
        "community_impact": "Include cheer zones, notify affected residents, and ensure
equitable route distribution.",
        "logistics_completeness": "Add timing chip details, plan course marshal positions,
and detail start/finish infrastructure.",
        "financial_viability": "Add budget breakdown with cost estimates and identify 3+
revenue sources.",
        "participant_experience": "Highlight scenic landmarks and detail post-race amenities
(medals, food, recovery).",
        "intent_alignment": "Review the user's original request and ensure the plan matches
their stated requirements.",
        "distance_compliance": "Verify the route is exactly 26.2 miles (42.195 km) for
marathon certification.",
    }
    return suggestions.get(criterion, f"Improve {criterion} (current score: {score:.2f})")

```

這裡的關鍵模式是**混合評估**：這項工具會先嘗試使用 LLM-as-Judge 指標進行 Vertex AI Eval，如果 API 無法使用，則會改用啟發式檢查。

連結評估工具代理

建立 `src/planner_agent/evaluator/agent.py`：

```

"""Evaluator Agent – scores marathon plans using Vertex AI Evaluation.

Uses gemini-3.1-pro-preview for best evaluation quality.
"""

import os

import vertexai
from google.adk.agents import LlmAgent
from google.adk.tools.preload_memory_tool import PreloadMemoryTool

from .prompts import INSTRUCTION
from .services.memory_manager import auto_save_memories
from .schemas import EvaluationResult

# Agent identity
AGENT_NAME = "evaluator_agent"
AGENT_DESCRIPTION = (
    "Evaluates marathon plans across multiple quality criteria using Vertex AI "
    "Evaluation with custom metrics. Acts as LLM-as-Judge to score plans and "
    "provide actionable feedback for iterative improvement."
)

# Model – use gemini-3.1-pro-preview for evaluation accuracy
MODEL = os.getenv("EVALUATOR_MODEL", "gemini-3.1-pro-preview")

# Structured output schema – guarantees every response matches EvaluationResult
OUTPUT_SCHEMA = EvaluationResult

# Criterion weights must sum to 1.0
CRITERION_WEIGHTS = {
    "safety_compliance": 0.20,
    "community_impact": 0.15,
    "logistics_completeness": 0.20,
    "financial_viability": 0.15,
    "participant_experience": 0.15,
    "intent_alignment": 0.10,
    "distance_compliance": 0.05,
}

SEVERITY_THRESHOLDS = {"high": 40.0, "medium": 60.0, "low": 80.0}

# Initialize Vertex AI
project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
location = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
if project_id:
    vertexai.init(project=project_id, location=location)

from google.genai.types import GenerateContentConfig, ThinkingConfig
from .tools import evaluate_plan

# Enable thinking for better evaluation reasoning

```

```
evaluator_config = GenerateContentConfig(max_output_tokens=4096)
if "pro" in MODEL:
    evaluator_config.thinking_config = ThinkingConfig(thinking_budget=1024)

evaluator_agent = LlmAgent(
    name="evaluator_agent",
    model=MODEL,
    description=AGENT_DESCRIPTION,
    static_instruction=INSTRUCTION,
    output_schema=OUTPUT_SCHEMA,
    generate_content_config=evaluator_config,
    include_contents='none',
    tools=[PreloadMemoryTool(), evaluate_plan],
    after_agent_callback=auto_save_memories,
)

root_agent = evaluator_agent
```

現在在 `src/planner_agent/evaluator/services/memory_manager.py` 建立評估人員的記憶體管理工具，追蹤評估記錄和評分趨勢：

```

"""Memory Manager for Evaluator Agent."""

import os
from typing import TYPE_CHECKING

from google.adk.memory import VertexAiMemoryBankService
from vertexai._genai.types import (
    MemoryBankCustomizationConfig,
    MemoryBankCustomizationConfigMemoryTopic as MemoryTopic,
    MemoryBankCustomizationConfigMemoryTopicCustomMemoryTopic as CustomMemoryTopic,
)

if TYPE_CHECKING:
    from google.adk.agents.callback_context import CallbackContext

EVALUATION_HISTORY = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="evaluation_history",
        description="""Track evaluation results: scores, pass/fail verdicts,
        findings count, and iteration numbers.""",
    )
)

SCORING_TRENDS = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="scoring_trends",
        description="""Track which criteria consistently score low and
        average scores by category.""",
    )
)

def create_memory_service(project=None, location=None, agent_engine_id=None):
    project = project or os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = location or os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
    agent_engine_id = agent_engine_id or os.environ.get("AGENT_ENGINE_ID")
    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT required")
    if not agent_engine_id:
        return None
    return VertexAiMemoryBankService(project=project, location=location,
agent_engine_id=agent_engine_id)

async def auto_save_memories(callback_context: "CallbackContext") -> None:
    agent_engine_id = os.environ.get("AGENT_ENGINE_ID")
    if not agent_engine_id:
        return
    try:
        svc = VertexAiMemoryBankService(
            project=os.environ.get("GOOGLE_CLOUD_PROJECT"),

```

```
        location=os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1"),
        agent_engine_id=agent_engine_id,
    )
    await svc.add_session_to_memory(callback_context._invocation_context.session)
except Exception as e:
    print(f"Warning: Failed to save memories: {e}")
```

最後，設定評估工具套件匯出作業。取代 `src/planner_agent/evaluator/__init__.py`：

```
try:
    from .agent import root_agent, AGENT_NAME, MODEL
    __all__ = ["root_agent", "AGENT_NAME", "MODEL"]
except Exception:
    __all__ = ["root_agent"]
```

重要概念：ThinkingConfig。評估人員會使用 `thinking_budget=1024`，讓 Gemini 先根據評估條件進行推理，再做出回應。這會提高評分準確度，但延遲時間會稍微增加。

步驟 5 · 約 12 分鐘

5. 建立 Simulation Controller 代理程式

模擬控制器是最後的把關者，會快速檢查先決條件，確認方案有足夠的資料可執行模擬，但不會重新評估品質。

定義核准結構定義

建立 `src/simulator_agent/agent/schemas.py`：

```
"""Schemas for the Simulation Controller Agent."""

from pydantic import BaseModel, Field

class SimulationApproval(BaseModel):
    """Structured output from the Simulation Controller Agent."""

    approved: bool = Field(description="Whether the plan is approved for simulation")
    overall_readiness: float = Field(ge=0.0, le=1.0, description="Readiness score (0-1)")
    route_feasibility: str = Field(description="'feasible', 'marginal', or 'infeasible'")
    logistics_readiness: str = Field(description="'ready', 'partial', or 'not_ready'")
    safety_clearance: str = Field(description="'cleared', 'conditional', or 'blocked'")
    blockers: list[str] = Field(default_factory=list, description="Blocking issues")
    recommendations: list[str] = Field(default_factory=list, description="Non-blocking suggestions")
    summary: str = Field(default="", description="Summary of the decision")
```

建立準備度檢查工具

建立 `src/simulator_agent/agent/tools.py` - 確定性檢查清單，不需要 LLM：


```

"""Tools for the Simulation Controller Agent."""

import re
from typing import Any

async def check_plan_readiness(plan_text: str) -> dict[str, Any]:
    """Check a marathon plan for simulation readiness.

    Performs deterministic checks on the plan text to identify
    missing elements and assess completeness.
    """
    plan_lower = plan_text.lower()

    checklist = {
        "distance_specified": bool(
            re.search(r'(\d+(?:\.\d+)?)\s*(?:miles?|mi|kilometers?|km)\b', plan_lower)
        ),
        "water_stations": "water station" in plan_lower,
        "medical_tents": any(kw in plan_lower for kw in ["medical tent", "first aid",
"medical station"]),
        "timing_system": any(kw in plan_lower for kw in ["timing", "chip", "tracking"]),
        "start_finish": any(kw in plan_lower for kw in ["start line", "finish line",
"start/finish", "starting area"]),
        "emergency_access": any(kw in plan_lower for kw in ["emergency", "ambulance",
"evacuation"]),
        "budget_included": any(kw in plan_lower for kw in ["budget", "cost", "revenue",
"expense"]),
        "timeline_included": any(kw in plan_lower for kw in ["schedule", "timeline", "race
day", "setup"]),
    }

    passed = sum(checklist.values())
    total = len(checklist)

    return {
        "checklist": checklist,
        "missing_elements": [k for k, v in checklist.items() if not v],
        "readiness_score": round(passed / total, 2),
        "passed_checks": passed,
        "total_checks": total,
    }

def get_tools() -> list:
    return [check_plan_readiness]

```

建立 ADK 技能

ADK Skills 提供代理在執行階段載入的**程序知識**。建立 `src/simulator_agent/skills/review-marathon-plan/SKILL.md`：

```

---
name: review-marathon-plan
description: Step-by-step methodology for reviewing marathon plans for simulation readiness,
covering route feasibility, logistics completeness, and safety clearance.
---

# Review Marathon Plan

## Purpose

Systematically confirm that a marathon plan is ready for simulation execution by verifying
the presence of all required prerequisite data.

## Procedure

### Step 1: Prerequisite Data Verification

Use the `check_plan_readiness` tool to confirm the presence of:

1. **Route Data**: Waypoints, landmarks, distance (26.2 mi / 42.195 km), start/finish
locations.
2. **Logistics Data**: Water stations, medical tents, timing systems, participant scale.
3. **Safety Data**: Emergency access routes, evacuation plans, crowd management.

### Step 2: Approval Decision

1. Set `approved=true` ONLY if all three data categories are present.
2. If any critical data is missing, set `approved=false` and list missing elements in
`blockers`.
3. Provide `recommendations` for minor data gaps.

```

為模擬控制器代理程式接線

建立 `src/simulator_agent/agent/config.py` :

```

"""Configuration for the Simulation Controller Agent."""

import os
from .schemas import SimulationApproval

AGENT_NAME = "simulator_agent"
AGENT_DESCRIPTION = (
    "Simulation Controller Agent. Reviews marathon plans for simulation readiness, "
    "assessing route feasibility, logistics completeness, and safety clearance."
)
MODEL = os.getenv("SIMULATOR_MODEL", "gemini-3-flash-preview")
OUTPUT_SCHEMA = SimulationApproval

```

建立 `src/simulator_agent/agent/prompts.py` :

```
"""Prompts for the Simulation Controller Agent."""
```

```
INSTRUCTION = """You are the Simulation Controller Agent – the final gatekeeper before a marathon plan enters simulation.
```

```
Your role is to perform a fast "Simulation Prerequisite Check" on marathon plans. You do NOT evaluate quality (the Evaluator Agent does that). You simply confirm the plan has all data required for the simulation engine.
```

```
## Available ADK Skills
```

```
1. review-marathon-plan – Check for simulation prerequisites (Route, Logistics, Safety).
```

```
## Prerequisite Check
```

1. **Route Data**: Waypoints, distance (26.2 mi / 42.195 km), start/finish locations.
2. **Logistics Data**: Timing and registration infrastructure, participant count.
3. **Safety Clearance**: Emergency access, evacuation plan, crowd management.

```
## Approval Decision
```

- **Approved** (``approved=true``): All prerequisite data present.
- **Rejected** (``approved=false``): Critical data missing. List in ``blockers``.

```
Always return a structured SimulationApproval."""
```

建立 `src/simulator_agent/agent/agent.py` :

```

"""Simulation Controller Agent – reviews plans for simulation readiness."""

import os
import pathlib

import vertexai
from google.adk.agents import LlmAgent
from google.adk.skills import load_skill_from_dir
from google.adk.tools.preload_memory_tool import PreloadMemoryTool
from google.adk.tools.skill_toolset import SkillToolset

from ..services.memory_manager import auto_save_memories
from .config import AGENT_DESCRIPTION, AGENT_NAME, MODEL, OUTPUT_SCHEMA
from .prompts import INSTRUCTION
from .tools import get_tools

# Initialize Vertex AI
project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
location = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
if project_id:
    vertexai.init(project=project_id, location=location)

# Load the review-marathon-plan ADK Skill
_skills_dir = pathlib.Path(__file__).parent.parent / "skills"
_review_skill = load_skill_from_dir(_skills_dir / "review-marathon-plan")
_skill_toolset = SkillToolset(skills=[_review_skill])

_tools = [_skill_toolset, PreloadMemoryTool(), *get_tools()]

from google.genai.types import GenerateContentConfig, ThinkingConfig

_agent_kwargs = dict(
    name=AGENT_NAME,
    model=MODEL,
    description=AGENT_DESCRIPTION,
    static_instruction=INSTRUCTION,
    tools=_tools,
    generate_content_config=GenerateContentConfig(
        thinking_config=ThinkingConfig(thinking_budget=0),
        max_output_tokens=4096,
    ),
    after_agent_callback=auto_save_memories,
)
if OUTPUT_SCHEMA is not None:
    _agent_kwargs["output_schema"] = OUTPUT_SCHEMA

root_agent = LlmAgent(**_agent_kwargs)

```

現在，請在 `src/simulator_agent/services/memory_manager.py` 建立模擬器的記憶體管理工具：

```

"""Memory Manager for Simulation Controller Agent."""

import os
from typing import TYPE_CHECKING

from google.adk.memory import VertexAiMemoryBankService
from vertexai._genai.types import (
    MemoryBankCustomizationConfig,
    MemoryBankCustomizationConfigMemoryTopic as MemoryTopic,
    MemoryBankCustomizationConfigMemoryTopicCustomMemoryTopic as CustomMemoryTopic,
)

if TYPE_CHECKING:
    from google.adk.agents.callback_context import CallbackContext

APPROVAL_HISTORY = MemoryTopic(
    custom_memory_topic=CustomMemoryTopic(
        label="approval_history",
        description="""Track approval decisions: verdicts, readiness scores, common
blockers."""),
)

def create_memory_service(project=None, location=None, agent_engine_id=None):
    project = project or os.environ.get("GOOGLE_CLOUD_PROJECT")
    location = location or os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
    agent_engine_id = agent_engine_id or os.environ.get("AGENT_ENGINE_ID")
    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT required")
    if not agent_engine_id:
        return None
    return VertexAiMemoryBankService(project=project, location=location,
agent_engine_id=agent_engine_id)

async def auto_save_memories(callback_context: "CallbackContext") -> None:
    agent_engine_id = os.environ.get("AGENT_ENGINE_ID")
    if not agent_engine_id:
        return
    try:
        svc = VertexAiMemoryBankService(
            project=os.environ.get("GOOGLE_CLOUD_PROJECT"),
            location=os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1"),
            agent_engine_id=agent_engine_id,
        )
        await svc.add_session_to_memory(callback_context._invocation_context.session)
    except Exception as e:
        print(f"Warning: Failed to save memories: {e}")

```

設定套件匯出作業。取代 `src/simulator_agent/agent/__init__.py` :

```
try:
    from .agent import root_agent
    from .config import AGENT_NAME, MODEL
    __all__ = ["root_agent", "AGENT_NAME", "MODEL"]
except Exception:
    pass
```

取代 `src/simulator_agent/__init__.py` :

```
try:
    from .agent import root_agent
    from .agent.config import AGENT_NAME, MODEL
    __all__ = ["root_agent", "AGENT_NAME", "MODEL"]
except Exception:
    pass
```

步驟 6 · 約 15 分鐘

6. 建構馬拉松規劃工具代理程式

馬拉松規劃工具是主要自動調度管理工具，這項工具會使用 ADK Skills 取得網域知識，將品質評估工作委派給 Evaluator 子代理程式，並透過 A2A 連線至 Simulation Controller。

定義規劃工具結構定義和設定

建立 `src/planner_agent/agent/schemas.py`：

```
"""Schemas for the Marathon Planner Agent."""

from pydantic import BaseModel, Field

class MarathonPlan(BaseModel):
    """Structured marathon plan output (used in Phase 2+)."""

    city: str = Field(description="City where the marathon takes place")
    date: str = Field(description="Event date in YYYY-MM-DD format")
    theme: str = Field(description="Marathon theme: scenic, fast, charity, etc.")
    participants: int = Field(ge=0, description="Expected number of participants")
    route_summary: str = Field(description="High-level route description")
    route_waypoints: list[str] = Field(default_factory=list, description="Ordered waypoints")
    distance_miles: float = Field(ge=0, description="Total route distance in miles")
    logistics_summary: str = Field(description="Key logistics overview")
    budget_summary: str = Field(description="Budget overview")
    key_risks: list[str] = Field(default_factory=list, description="Top risks and mitigations")
```

建立 `src/planner_agent/agent/config.py`：

```
"""Configuration for the Marathon Planner Agent."""

import os

AGENT_NAME = "planner_agent"
AGENT_DESCRIPTION = (
    "Marathon Planner Agent — Lead Architect. "
    "Designs comprehensive city marathon plans with built-in expertise in route design, "
    "traffic management, community impact, and economics. Evaluates plans via the "
    "Evaluator Agent (A2A) and submits for approval to the Simulation Controller (A2A). "
)
MODEL = os.getenv("PLANNER_MODEL", "gemini-3-flash-preview")
# Phase 1: No structured output — agent returns free-form text
OUTPUT_SCHEMA = None
```

撰寫規劃工具指令

建立 `src/planner_agent/agent/planner-instruction.md`：

```

# Role
Marathon Planner Agent (city marathon event architect).
Goal: Design comprehensive marathon plan based on user constraints.

# ADK Skills (LOAD ONCE BEFORE PLANNING)
1. `route-planning`: Generate route using `plan_marathon_route`.
2. `plan-evaluation`: Analyze demographics, capacity, revenue, safety.

# Core Requirements
- Safety: Emergency corridors, traffic cover.
- Community: Local business, noise, inclusivity.
- Logistics: Start/finish capacity, restrooms, roads.
- Finances: Maximize revenue/sponsorships.
- Experience: Scenic, runner comfort.

# User Prerequisites
Clarify if missing: City, Date/Season, Theme, Scale (participants), Budget, Special constraints.

# Deliverables
1. Route Design: GeoJSON via `plan_marathon_route` tool.
2. Traffic: Closures, detours, mitigation.
3. Community: Engagement, cheer zones, noise.
4. Economics: Revenue, costs, sponsors.
5. Logistics: Porta-potties, capacity, timing.
6. Timeline: Setup to teardown, waves.
7. Risks: Weather, crowd, emergency.

# A2A Collaboration
1. **Evaluator (`evaluator_agent`)**:
  - Send plan for 7-criteria scoring.
  - SINGLE PASS ONLY. Do not call twice to verify successful fixes.
2. **Simulation Controller (`simulator_agent`)**:
  - Call once after evaluation completes (REGARDLESS OF SCORE).
  - Accept result, DO NOT call again.

# Workflow
1. Gather reqs.
2. Load skills (ONCE).
3. Call `plan_marathon_route`.
4. Complete design.
5. Send to Evaluator.
6. Send to Simulator.
7. Present final.

# Rules & Format
- Personality: Pragmatic, detail-oriented.

```

建立 `src/planner_agent/agent/prompts.py` :


```

"""System instructions for the Marathon Planner Agent."""

import pathlib

_PROMPT_DIR = pathlib.Path(__file__).parent
INSTRUCTION = (_PROMPT_DIR / "planner-instruction.md").read_text()

```

建立驗證模組

如要對 Agent Engine 進行 A2A 呼叫，代理程式需要 Google Cloud 授權。建立

`src/planner_agent/agent/auth.py`：

```

"""Authentication utilities for Agent Engine A2A connections.

Provides Google Cloud auth for httpx requests to Agent Engine agents.
"""

import httpx
from google.auth import default
from google.auth.credentials import Credentials
from google.auth.transport.requests import Request as AuthRequest

class GoogleAuthRefresh(httpx.Auth):
    """Google Cloud Auth with lazy credential refresh.

    Credentials are initialized lazily on first request to avoid
    serialization issues with ADK agents.
    """

    def __init__(self, scopes: list[str] | None = None) -> None:
        self.scopes = scopes or ["https://www.googleapis.com/auth/cloud-platform"]
        self.credentials: Credentials | None = None
        self.auth_request: AuthRequest | None = None

    def auth_flow(self, request: httpx.Request):
        if self.credentials is None:
            self.credentials, _ = default(scopes=self.scopes)
            self.auth_request = AuthRequest()

        if not self.credentials.valid:
            self.credentials.refresh(self.auth_request)

        request.headers["Authorization"] = f"Bearer {self.credentials.token}"
        yield request

```

新增 ADK 技能

建立 `src/planner_agent/skills/route-planning/SKILL.md`：

```

---
name: route-planning
description:
  Generates high-fidelity marathon routes using road network data (Dijkstra's algorithm)
  and outputting GeoJSON for visualization.
---

# Route Planning Skill

**Goal:** Design a mathematically perfect 42.195 km marathon route using real road network
data.

## Capabilities

- **Automated Route Generation:** Uses a built-in road network and Dijkstra's algorithm to
calculate a certified 42.195 km route between specified landmarks.
- **GeoJSON Output:** Returns a standards-compliant GeoJSON FeatureCollection.

## Resources

### Tools (Python)
- `tools.py`: Contains the `plan_marathon_route`, `add_water_stations`, and
`add_medical_tents` implementations.

### References
- `references/marathon_planning_guide.md`: Marathon standards, road width, traffic severity,
landmarks.

```

建立 `src/planner_agent/skills/plan-evaluation/SKILL.md` :

```

---
name: plan-evaluation
description:
  Evaluates the plan against financial, community, and logistical goals.
---

# Plan Evaluation Skill

**Goal:** Provide verification data for the event including capacity, budget, demographics,
and nuisance checking.

## Resources

### References
- `references/evaluation_criteria.md`: Detailed scoring rubrics.

```

ADK Skills 可讓您封裝代理在執行階段透過 `SkillToolset` 載入的網域知識 (參考文件、資料檔案、指令碼)。這樣一來，指令不會過於冗長，但服務專員仍能獲得深入的程序知識。

接線規劃工具

建立 `src/planner_agent/agent/tools.py`，代理程式會在此連線：

```
"""Tools for the Marathon Planner Agent.
```

```
Contains:
```

- A2A infrastructure (SerializableRemoteA2aAgent, URL helpers)
 - Remote A2A agent creators for Evaluator and Simulation Controller
 - get_tools() - assembles all tools including SkillToolset
- ```
"""
```

```
import logging
import os
```

```
import httpx
from google.adk.agents.remote_a2a_agent import RemoteA2aAgent
from google.adk.tools.agent_tool import AgentTool
from google.adk.tools.function_tool import FunctionTool
from a2a.client.client import ClientConfig as A2AClientConfig
from a2a.client.client_factory import ClientFactory as A2AClientFactory
from a2a.types import TransportProtocol as A2ATransport
```

```
from .auth import GoogleAuthRefresh
from ..evaluator.agent import root_agent as evaluator_agent
```

```
logger = logging.getLogger(__name__)
```

```
AGENT_TIMEOUT_SECONDS = 120
```

```
=====
A2A INFRASTRUCTURE
=====
```

```
def _get_agent_a2a_endpoint(resource_name: str, default_port: int = 8080) -> str:
 """Construct A2A card endpoint URL from resource name or local address."""
 if resource_name.startswith("local"):
 port = resource_name.split(":")[1] if ":" in resource_name else default_port
 return f"http://127.0.0.1:{port}/.well-known/agent-card.json"

 parts = resource_name.split("/")
 try:
 location = parts[parts.index("locations") + 1]
 return f"https://{location}-
aiplatform.googleapis.com/v1beta1/{resource_name}/a2a/v1/card"
 except (ValueError, IndexError):
 return resource_name
```

```
def _get_agent_a2a_url(resource_name: str) -> str | None:
 """Construct the regional A2A message URL from Agent Engine resource name."""
 if resource_name.startswith("local"):
 return None
 parts = resource_name.split("/")
```

```

location = parts[parts.index("locations") + 1]
return f"https://{location}-aiplatform.googleapis.com/v1beta1/{resource_name}/a2a"

```

```

class SerializableRemoteA2aAgent(RemoteA2aAgent):

```

```

 """RemoteA2aAgent with authentication and Agent Engine URL fix.

```

```

 Handles two Agent Engine issues:

```

1. Creates httpx client lazily with Google Cloud auth
  2. Fixes agent card URL (Agent Engine returns global URL that 404s)
- ```

    """

```

```

def __init__(self, *, a2a_url: str | None = None, **kwargs):

```

```

    super().__init__(**kwargs)

```

```

    self._a2a_url_override = a2a_url

```

```

async def _ensure_httpx_client(self) -> httpx.AsyncClient:

```

```

    if self._httpx_client is None:

```

```

        self._httpx_client = httpx.AsyncClient(
            timeout=httpx.Timeout(timeout=AGENT_TIMEOUT_SECONDS),
            headers={"Content-Type": "application/json"},
            auth=GoogleAuthRefresh(),
        )

```

```

        self._httpx_client_needs_cleanup = True

```

```

    if self._a2a_client_factory is None:

```

```

        self._a2a_client_factory = A2aClientFactory(
            config=A2aClientConfig(
                httpx_client=self._httpx_client,
                streaming=False, polling=False,
                supported_transports=[A2aTransport.http_json, A2aTransport.jsonrpc],
            )
        )

```

```

    return self._httpx_client

```

```

async def _resolve_agent_card_from_url(self, url: str):

```

```

    card = await super()._resolve_agent_card_from_url(url)

```

```

    if self._a2a_url_override:

```

```

        logger.info(f"Overriding agent card URL: {card.url} → {self._a2a_url_override}")

```

```

        card.url = self._a2a_url_override

```

```

    return card

```

```

def create_evaluator_tool() -> AgentTool:

```

```

    """Create Evaluator Agent tool – local sub-agent, no A2A needed."""

```

```

    return AgentTool(agent=evaluator_agent)

```

```

def create_simulator_agent() -> RemoteA2aAgent:

```

```

    """Create remote connection to Simulation Controller Agent via A2A."""

```

```

    resource_name = os.environ.get("SIMULATOR_AGENT_RESOURCE_NAME")

```

```

    if not resource_name:

```

```

        raise ValueError("SIMULATOR_AGENT_RESOURCE_NAME environment variable must be set")

    endpoint = _get_agent_a2a_endpoint(resource_name, default_port=8089)
    logger.info(f"Creating Simulation Controller Agent connection: {endpoint}")

    return SerializableRemoteA2aAgent(
        name="simulator_agent",
        description=(
            "Simulation Controller Agent for marathon plans. "
            "Reviews plans for simulation readiness, assessing route feasibility, "
            "logistics completeness, and safety clearance."
        ),
        agent_card=endpoint,
        a2a_url=_get_agent_a2a_url(resource_name),
    )

def create_simulation_controller_tool() -> AgentTool:
    return AgentTool(agent=create_simulator_agent())

# =====
# TOOL EXPORT
# =====

def get_tools() -> list:
    """Return the tools for the Marathon Planner Agent."""
    import pathlib
    import importlib.util
    from google.adk.skills import load_skill_from_dir
    from google.adk.tools.preload_memory_tool import PreloadMemoryTool
    from google.adk.tools.skill_toolset import SkillToolset

    # Load ADK Skills
    skills_dir = pathlib.Path(__file__).parent.parent / "skills"
    skills = [
        load_skill_from_dir(skills_dir / name)
        for name in sorted(skills_dir.iterdir())
        if name.is_dir() and not name.name.startswith("_")
    ]
    skill_toolset = SkillToolset(skills=skills)

    # Load route-planning tools dynamically (handles hyphenated directory names)
    def load_tool_from_skill(skill_name, tool_name):
        skill_tool_path = skills_dir / skill_name / "tools.py"
        if not skill_tool_path.exists():
            return None
        try:
            spec = importlib.util.spec_from_file_location(f"{skill_name}.tools",
skill_tool_path)
            module = importlib.util.module_from_spec(spec)

```

```

        spec.loader.exec_module(module)
        return getattr(module, tool_name, None)
    except Exception as e:
        logger.error(f"Error loading tool {tool_name} from {skill_name}: {e}")
        return None

plan_marathon_route_func = load_tool_from_skill("route-planning", "plan_marathon_route")

tools = [skill_toolset, PreloadMemoryTool()]

if plan_marathon_route_func:
    tools.append(FunctionTool(func=plan_marathon_route_func))

# Add local evaluator sub-agent
tools.append(create_evaluator_tool())
logger.info("Added local Evaluator tool")

# Add remote Simulation Controller via A2A (if configured)
if os.environ.get("SIMULATOR_AGENT_RESOURCE_NAME"):
    tools.append(create_simulation_controller_tool())
    logger.info("Added A2A Simulation Controller tool")

return tools

```

`get_tools()` 函式會顯示三種不同的工具模式：

1. **SkillToolset**：從 ADK Skills 載入程序知識
2. **AgentTool**：將評估工具包裝為本機子代理
3. **SerializableRemoteA2aAgent**：透過 A2A 通訊協定連結模擬器

連結規劃工具代理程式

建立 `src/planner_agent/agent/agent.py`：

```

"""Marathon Planner Agent - LlmAgent wiring.

Wires config + prompts + tools + skills into the LlmAgent.
"""

import os

import vertexai
from google.adk.agents import LlmAgent

from .config import AGENT_NAME, AGENT_DESCRIPTION, MODEL, OUTPUT_SCHEMA
from .prompts import INSTRUCTION
from .tools import get_tools
from ..services.memory_manager import auto_save_memories

# Initialize Vertex AI
project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
location = os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
if project_id:
    vertexai.init(project=project_id, location=location)

from google.genai.types import GenerateContentConfig, ThinkingConfig

agent_kwargs = dict(
    name=AGENT_NAME,
    model=MODEL,
    description=AGENT_DESCRIPTION,
    static_instruction=INSTRUCTION,
    tools=get_tools(),
    generate_content_config=GenerateContentConfig(
        thinking_config=ThinkingConfig(thinking_budget=2048),
    ),
    after_agent_callback=auto_save_memories,
)

if OUTPUT_SCHEMA is not None:
    agent_kwargs["output_schema"] = OUTPUT_SCHEMA

root_agent = LlmAgent(**agent_kwargs)

```

設定套件匯出作業。取代 `src/planner_agent/agent/__init__.py` :

```

from .agent import root_agent
from .config import AGENT_NAME, MODEL, AGENT_DESCRIPTION, OUTPUT_SCHEMA

__all__ = ["root_agent", "AGENT_NAME", "MODEL", "AGENT_DESCRIPTION", "OUTPUT_SCHEMA"]

```

取代 `src/planner_agent/__init__.py` :


```
try:
    from .agent import root_agent
    from .agent.config import AGENT_NAME, MODEL
    __all__ = ["root_agent", "AGENT_NAME", "MODEL"]
except Exception:
    __all__ = ["root_agent"]
```

ThinkingConfig 預算：規劃工具使用 `thinking_budget=2048` 進行深入規劃，評估工具使用 `1024` 進行評估推論，模擬工具則使用 `0` (快速檢查清單不需要思考)。根據任務複雜度調整預算。

步驟 7 · 約 15 分鐘

7. 部署及測試多代理系統

現在，請將所有項目整合在一起：為每個代理建立 A2A 伺服器，然後測試 Solo 模式 (Planner + Evaluator) 和 Full Team 模式 (所有三個代理)。

建立規劃工具的 A2A 代理資訊卡

建立 `src/planner_agent/runtime/agent_card.py`：

```
"""A2A Agent Card for Marathon Planner Agent."""

from a2a.types import AgentCapabilities, AgentCard, AgentSkill
from vertexai.preview.reasoning_engines.templates.a2a import create_agent_card

def create_marathon_planner_card() -> AgentCard:
    skill = AgentSkill(
        id="plan_marathon",
        name="Plan City Marathon",
        description=(
            "Design a comprehensive city marathon plan by coordinating with "
            "specialist agents. Evaluates plan quality and returns an actionable plan."
        ),
        tags=["marathon", "planning", "orchestration", "multi-agent"],
        examples=[
            "Plan a scenic marathon through Las Vegas for 30,000 runners",
            "Design a charity marathon in Austin for October 2026",
        ],
    )

    card = create_agent_card(
        agent_name="planner_agent",
        description=(
            "Marathon Planner Agent – Lead Orchestrator that designs city marathon "
            "plans. Coordinates specialist agents via A2A. Evaluates plans with "
            "Vertex AI Eval. Powered by Gemini 3 Flash Preview."
        ),
        skills=[skill],
    )

    if card.capabilities is None:
        card.capabilities = AgentCapabilities(streaming=True)
    else:
        card.capabilities.streaming = True

    return card
```

建立規劃工具的 A2A 執行器

建立 `src/planner_agent/runtime/agent_executor.py`：

```

"""A2A Agent Executor for Marathon Planner Agent."""

import json
import os

import vertexai
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.tasks import TaskUpdater
from a2a.types import TaskState, TextPart, UnsupportedOperationError
from a2a.utils import new_agent_text_message
from a2a.utils.errors import ServerError
from google.adk import Runner
from google.genai import types

from ..services.memory_manager import create_memory_service
from ..services.session_manager import SessionManager, create_session_service


class MarathonPlannerExecutor(AgentExecutor):
    """Execute requests via A2A protocol."""

    def __init__(self):
        self.agent = None
        self.runner = None
        self.session_manager: SessionManager | None = None

    def _init_agent(self) -> None:
        if self.agent is None:
            try:
                from planner_agent.agent import root_agent
            except ImportError:
                from ..agent import root_agent

            project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
            location = os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
            vertexai.init(project=project_id, location=location)
            self.agent = root_agent

        if self.runner is None:
            project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
            location = os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")

            session_service = create_session_service(project=project_id, location=location)
            memory_service = create_memory_service(project=project_id, location=location)

            from google.adk.apps import App
            from google.adk.agents.context_cache_config import ContextCacheConfig

            app = App(

```

```

        name=self.agent.name,
        root_agent=self.agent,
        context_cache_config=ContextCacheConfig(
            cache_intervals=10, ttl_seconds=3600, min_tokens=4096,
        )
    )
    self.runner = Runner(
        app=app,
        session_service=session_service,
        memory_service=memory_service,
    )

    if self.session_manager is None:
        self.session_manager = SessionManager(
            session_service=self.runner.session_service,
        )

    async def execute(self, context: RequestContext, event_queue: EventQueue) -> None:
        if self.agent is None:
            self._init_agent()

        user_id = (
            context.message.metadata.get("user_id")
            if context.message and context.message.metadata
            else "marathon_user"
        )

        updater = TaskUpdater(event_queue, context.task_id, context.context_id)
        if not hasattr(context, "current_task") or not context.current_task:
            await updater.submit()
        await updater.start_work()

        request_data = context.get_user_input()
        if not request_data:
            await updater.update_status(TaskState.failed, message=new_agent_text_message("No
request data"), final=True)
            return

        try:
            from google.adk.runners import RunConfig
            from google.adk.agents.run_config import ToolThreadPoolConfig
            from google.genai.types import ContextWindowCompressionConfig

            await updater.update_status(TaskState.working,
message=new_agent_text_message("Planning marathon..."))

            session_id = await self.session_manager.get_or_create_session(
                context_id=context.context_id, app_name=self.runner.app_name,
user_id=user_id,
            )

            content = types.Content(role="user", parts=[types.Part(text=request_data)])

```

```

        final_event = None
    async for event in self.runner.run_async(
        session_id=session_id, user_id=user_id, new_message=content,
        run_config=RunConfig(
            tool_thread_pool_config=ToolThreadPoolConfig(max_workers=4),
            max_llm_calls=15,
            context_window_compression=ContextWindowCompressionConfig(),
        )
    ):
        if event.is_final_response():
            final_event = event

    if final_event and final_event.content and final_event.content.parts:
        text = "".join(p.text for p in final_event.content.parts if hasattr(p,
"text") and p.text)
        if text:
            await updater.add_artifact([TextPart(text=text)], name="marathon_plan")
            await updater.complete()
            return

        await updater.update_status(TaskState.failed,
message=new_agent_text_message("Failed to generate plan"), final=True)
        except Exception as e:
            await updater.update_status(TaskState.failed,
message=new_agent_text_message(f"Planning failed: {e}"), final=True)

    async def cancel(self, context: RequestContext, event_queue: EventQueue) -> None:
        raise ServerError(error=UnsupportedOperationError())

```

建立本機 A2A 伺服器

建立 `src/planner_agent/runtime/local_server.py` :

```
"""Local A2A server for the Marathon Planner Agent.
```

```
Usage: uv run python -m src.planner_agent.runtime.local_server
"""
```

```
import os
from dotenv import load_dotenv
load_dotenv()
os.environ["GOOGLE_GENAI_USE_VERTEXAI"] = "true"

import asyncio
import uvicorn
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore
from a2a.types import TransportProtocol
from google.adk import Runner
from google.adk.a2a.executor.a2a_agent_executor import A2aAgentExecutor,
A2aAgentExecutorConfig
from google.adk.sessions import InMemorySessionService

AGENT_PORT = 8084

def create_marathon_planner_server():
    from ..agent import root_agent
    from .agent_card import create_marathon_planner_card

    runner = Runner(
        app_name=root_agent.name,
        agent=root_agent,
        session_service=InMemorySessionService(),
    )

    executor = A2aAgentExecutor(runner=runner, config=A2aAgentExecutorConfig())
    handler = DefaultRequestHandler(agent_executor=executor, task_store=InMemoryTaskStore())

    card = create_marathon_planner_card()
    card.url = f"http://localhost:{AGENT_PORT}"
    card.preferred_transport = TransportProtocol.jsonrpc

    return A2AStarletteApplication(agent_card=card, http_handler=handler)

async def run_server():
    project = os.environ.get("GOOGLE_CLOUD_PROJECT")
    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT must be set. Check .env file.")

    print("=" * 60)
    print("Starting Marathon Planner Agent Local A2A Server")
    print(f"Project: {project}")
```

```

print("=" * 60)

app = create_marathon_planner_server()
config = uvicorn.Config(app.build(), host="127.0.0.1", port=AGENT_PORT, log_level="info",
loop="none")

print(f"Planner A2A server: http://127.0.0.1:{AGENT_PORT}")
print(f"Agent card: http://127.0.0.1:{AGENT_PORT}/.well-known/agent-card.json")
print("Press Ctrl+C to stop")
print("=" * 60)

server = uvicorn.Server(config)
await server.serve()

if __name__ == "__main__":
    asyncio.run(run_server())

```

建立 `src/simulator_agent/runtime/agent_card.py` :

```

"""A2A Agent Card for Simulation Controller Agent."""

from a2a.types import AgentCard, AgentSkill
from vertexai.preview.reasoning_engines.templates.a2a import create_agent_card

def create_simulation_controller_card() -> AgentCard:
    skill = AgentSkill(
        id="review_marathon_plan",
        name="Review Marathon Plan",
        description="Review a marathon plan for simulation readiness.",
        tags=["simulation", "review", "approval", "marathon"],
    )
    return create_agent_card(
        agent_name="simulator_agent",
        description="Simulation Controller Agent - Reviews marathon plans for simulation readiness.",
        skills=[skill],
    )

```

按照與規劃工具執行器相同的模式建立 `src/simulator_agent/runtime/agent_executor.py` (更新 `simulator_agent` 的匯入項目) :

```

"""A2A Agent Executor for Simulation Controller Agent."""

import os
import vertexai
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.tasks import TaskUpdater
from a2a.types import TaskState, TextPart, UnsupportedOperationError
from a2a.utils import new_agent_text_message
from a2a.utils.errors import ServerError
from google.adk import Runner
from google.genai import types

from ..services.memory_manager import create_memory_service
from ..services.session_manager import SessionManager, create_session_service


class SimulationControllerExecutor(AgentExecutor):
    def __init__(self):
        self.agent = None
        self.runner = None
        self.session_manager = None

    def _init_agent(self):
        if self.agent is None:
            try:
                from simulator_agent.agent import root_agent
            except ImportError:
                from ..agent.agent import root_agent
            project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
            location = os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
            vertexai.init(project=project_id, location=location)
            self.agent = root_agent

        if self.runner is None:
            project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
            location = os.environ.get("AGENT_ENGINE_LOCATION") or
os.environ.get("GOOGLE_CLOUD_LOCATION", "us-central1")
            session_service = create_session_service(project=project_id, location=location)
            memory_service = create_memory_service(project=project_id, location=location)

            from google.adk.apps import App
            from google.adk.agents.context_cache_config import ContextCacheConfig
            app = App(name=self.agent.name, root_agent=self.agent,
                    context_cache_config=ContextCacheConfig(cache_intervals=10,
ttl_seconds=3600, min_tokens=4096))
            self.runner = Runner(app=app, session_service=session_service,
memory_service=memory_service)

        if self.session_manager is None:
            self.session_manager =

```



```

SessionManager(session_service=self.runner.session_service)

    async def execute(self, context: RequestContext, event_queue: EventQueue):
        if self.agent is None:
            self._init_agent()
            user_id = context.message.metadata.get("user_id") if context.message and
context.message.metadata else "planner_agent"
            updater = TaskUpdater(event_queue, context.task_id, context.context_id)
            if not hasattr(context, "current_task") or not context.current_task:
                await updater.submit()
            await updater.start_work()
            plan_data = context.get_user_input()
            if not plan_data:
                await updater.update_status(TaskState.failed, message=new_agent_text_message("No
plan data"), final=True)
                return
            try:
                await updater.update_status(TaskState.working,
message=new_agent_text_message("Reviewing plan..."))
                session_id = await self.session_manager.get_or_create_session(
                    context_id=context.context_id, app_name=self.runner.app_name,
user_id=user_id)
                content = types.Content(role="user", parts=[types.Part(text=plan_data)])
                final_event = None
                async for event in self.runner.run_async(session_id=session_id, user_id=user_id,
new_message=content):
                    if event.is_final_response():
                        final_event = event
                    if final_event and final_event.content and final_event.content.parts:
                        text = "".join(p.text for p in final_event.content.parts if hasattr(p,
"text") and p.text)
                        if text:
                            await updater.add_artifact([TextPart(text=text)], name="result")
                            await updater.complete()
                            return
                await updater.update_status(TaskState.failed,
message=new_agent_text_message("Failed"), final=True)
            except Exception as e:
                await updater.update_status(TaskState.failed,
message=new_agent_text_message(f"Review failed: {e}"), final=True)

    async def cancel(self, context, event_queue):
        raise ServerError(error=UnsupportedOperationError())

```

建立 `src/simulator_agent/runtime/local_server.py` :

```

"""Local A2A server for the Simulation Controller Agent.

Usage: uv run python -m src.simulator_agent.runtime.local_server
"""

import asyncio
import logging
import os

os.environ["GOOGLE_GENAI_USE_VERTEXAI"] = "true"

import uvicorn
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore
from dotenv import load_dotenv

load_dotenv()

AGENT_PORT = 8089

async def run_server():
    from .agent_card import create_simulation_controller_card
    from .agent_executor import SimulationControllerExecutor

    project = os.environ.get("GOOGLE_CLOUD_PROJECT")
    if not project:
        raise ValueError("GOOGLE_CLOUD_PROJECT must be set.")

    print("=" * 60)
    print("Starting Simulation Controller Agent Local A2A Server")
    print("=" * 60)

    agent_card = create_simulation_controller_card()
    executor = SimulationControllerExecutor()
    handler = DefaultRequestHandler(agent_executor=executor, task_store=InMemoryTaskStore())

    app = A2AStarletteApplication(agent_card=agent_card, http_handler=handler)
    config = uvicorn.Config(app.build(), host="127.0.0.1", port=AGENT_PORT, log_level="info",
loop="none")

    print(f"Simulator A2A server: http://127.0.0.1:{AGENT_PORT}")
    print(f"Agent card: http://127.0.0.1:{AGENT_PORT}/.well-known/agent.json")
    print("Press Ctrl+C to stop")
    print("=" * 60)

    server = uvicorn.Server(config)
    await server.serve()

```

```
if __name__ == "__main__":
    asyncio.run(run_server())
```

測試單人模式 - 規劃工具 + 評估工具

在單人模式中，規劃工具會將評估人員當做本機子代理程式。不需要 A2A。

啟動規劃工具伺服器：

```
uv run python -m src.planner_agent.runtime.local_server
```

畫面會顯示類似以下的輸出：

```
=====
Starting Marathon Planner Agent Local A2A Server
Project: your-project-id
=====
Planner A2A server: http://127.0.0.1:8084
Agent card: http://127.0.0.1:8084/.well-known/agent-card.json
Press Ctrl+C to stop
```

開啟新的 Cloud Shell 分頁，並傳送測試要求：

```
curl -s http://127.0.0.1:8084/.well-known/agent-card.json | python3 -m json.tool
```

畫面上應該會顯示代理程式資訊卡，其中包含 `planner_agent` 名稱和 `plan_marathon` 技能。

測試「完整團隊」模式：透過 A2A 取得所有三位代理人

在完整團隊模式中，模擬控制器會以獨立的 A2A 伺服器執行。

在第二個 Cloud Shell 分頁中啟動模擬器伺服器：

```
uv run python -m src.simulator_agent.runtime.local_server
```

然後在第一個分頁中，重新啟動 Planner 並連線至模擬器：

```
SIMULATOR_AGENT_RESOURCE_NAME=local:8089 uv run python -m
src.planner_agent.runtime.local_server
```

記錄中應會顯示：

```
Added local Evaluator tool
Added A2A Simulation Controller tool
```

現在 Planner 已連結至這兩個代理程式。收到馬拉松規劃要求時，這項功能會：

1. 使用 ADK Skills 設計企劃書

2. 委派給「評估者」(透過 `AgentTool` 的本機子代理)
3. 傳送至**模擬控制器**(透過 A2A 的遠端代理程式)
4. 提交最終計畫，並附上評估分數和模擬核准結果

同一套程式碼，但設定不同。單人模式與完整團隊模式只是環境變數切換鈕 `SIMULATOR_AGENT_RESOURCE_NAME`，規劃師代碼不會變更。

步驟 8 · 約 2 分鐘

8. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取費用，請清理本程式碼研究室建立的資源。

在每個 Cloud Shell 分頁中按下 `Ctrl+C`，停止所有正在執行之本機伺服器。

由於本程式碼研究室只使用 Vertex AI API 呼叫 (沒有 Cloud Run 服務或 Agent Engine 部署作業等持續性雲端資源)，因此不需要刪除任何雲端資源。

如果您是為了這個程式碼研究室建立專案，可以[刪除專案](#)：

```
gcloud projects delete $PROJECT_ID
```

9. 恭喜

恭喜！您已建構多代理馬拉松規劃系統，其中三個 AI 代理會透過 A2A 通訊協定協同合作，並由 Vertex AI Evaluation 提供品質保證。

目前所學內容

- 如何使用 **Agent Development Kit (ADK)** 搭配 `LlmAgent`、`ThinkingConfig` 和結構化 Pydantic 輸出內容建立 AI 代理
 - 如何使用 `MetricPromptBuilder` 建構自訂 **Vertex AI** 評估指標，以進行 LLM-as-Judge 評分和確定性檢查
 - 如何使用 `AgentTool` 將代理程式連線為**子代理程式**，以進行本機委派
 - 如何使用 `RemoteA2aAgent` 和 `A2AStarletteApplication`，透過 **A2A 通訊協定**連線至遠端代理
 - 如何使用 **ADK Skills** (`SkillToolset`) 取得程序知識，以及使用 **Memory Bank** (`PreloadMemoryTool`) 進行跨工作階段學習
-